

数字后端自动化 Agent 实验报告

作业：基于 Claude Code 的数字后端自动化 Agent

小组成员：曹闰程、杜昊、汤泳希

题目要求

核心任务：本次作业要求你以第一次作业中已经生成并验证通过的 RTL 为输入，围绕 Claude Code 构建一条基于 sky130hd 平台的数字后端自动化Agent。

你需要：

- 开发 MCP Server，将后端工具链的 Docker 操作封装为 Claude Code 可调用的工具；
- 编写 Skill，编排多阶段后端流程的执行逻辑；
- 配置 Rules 和 Hooks，定义流程规范与自动化检查。

最终目标：用户在 Claude Code 中给出一个已验证 RTL 设计及必要顶层信息后，Agent 应尽可能自动完成输入检查、约束生成、配置生成、工具调用、日志分析和报告输出。

1. 环境搭建说明

1.1 宿主机环境

项目	版本/说明
操作系统	Ubuntu 22.04 (WSL2, Windows11)
Docker	使用老师指定OpenROAD docker，用于运行 ORFS 容器
Node.js	≥ 18，用于运行 MCP Server
Claude Code	CLI 工具，使用claude-sonnet-4-6 模型

1.2 Docker 容器搭建

使用官方 `openroad/orfs:latest` 镜像，该镜像内置：

- Yosys 0.63+**：逻辑综合
- OpenROAD**：布图规划、布局、CTS、布线
- KLayout**：GDSII 生成与 DRC 检查
- sky130hd PDK**：SkyWater 130nm 高密度标准单元库

容器创建命令（一次性执行）：

```
docker run -d --name orfs-agent \
-v $(pwd)/design:/workspace/design \
openroad/orfs:latest sleep infinity
```

- 容器以 `sleep infinity` 常驻后台，避免每次重启开销。
- 宿主机 `./design/` 目录挂载至容器 `/workspace/design/`，实现文件双向共享。

1.3 MCP Server 启动

```
cd mcp_server
npm install
node src/index.js # 由 Claude Code 通过 stdio 自动管理
```

MCP Server 配置在 `.claude/settings.local.json` 中声明，Claude Code 启动时自动连接。

2. 工艺平台与输入文件说明

2.1 工艺平台

参数	值
工艺节点	SkyWater 130nm
平台标识	sky130hd (高密度标准单元)
目标库	sky130_fd_sc_hd__tt_025C_1v80
工作条件	TT corner, 25°C, 1.8V

2.2 RTL 源文件 (`design/rtl/alu.sv`)

按照作业要求，我们使用由上一个前端Agnest生成的SystemVerilog代码，代码实现了一个具有 8 种操作的ALU，代码如下：

```
module ALU(
    input      clock, reset,
    input [31:0] io_a, io_b,
    input [2:0] io_op,
    output [31:0] io_result,
    output      io_zero
);
    wire [7:0][31:0] _GEN = {
        {32'h0}, {32'h0},
        {{31'h0, $signed(io_a) < $signed(io_b)}}}, // SLT
        {io_a ^ io_b}, // XOR
        {io_a | io_b}, // OR
        {io_a & io_b}, // AND
        {io_a - io_b}, // SUB
        {io_a + io_b}}; // ADD
    assign io_result = _GEN[io_op];
    assign io_zero   = _GEN[io_op] == 32'h0;
endmodule
```

ALU 为**纯组合逻辑**，虽有 `clock/reset` 端口（Chisel 框架生成），但内部无寄存器，这对 CTS 阶段有直接影响，在后续自动化流程中会额外处理。

2.3 时序约束（`design/constraint.sdc`）

```
create_clock -name clock -period 10.0 [get_ports clock]
set_input_delay 2.0 -clock clock [get_ports {reset io_a io_b io_op}]
set_output_delay 2.0 -clock clock [get_ports {io_result io_zero}]
```

时钟周期 10 ns（100 MHz），输入/输出延迟各 2 ns，有效组合逻辑预算 6 ns，整体约束较为松弛。

2.4 流程配置（`design/config.mk`）

```
export DESIGN_NAME = ALU
export PLATFORM     = sky130hd
export VERILOG_FILES = /workspace/design/rtl/alu.sv
export SDC_FILE      = /workspace/design/constraint.sdc
export DIE_AREA      = 0 0 300 300
export CORE_AREA     = 10 10 290 290
export PLACE_DENSITY = 0.3
export PNR_TOOL       = openroad
```

Die 面积 300×300 μm，Core 面积 280×280 μm，布局密度 0.3，整体约束较为松弛

3. Agent 配置说明

3.1 整体架构



3.2 MCP Server 工具详解 (mcp_server/src/index.js)

MCP Server 基于 @modelcontextprotocol/sdk 实现，通过 stdio 与 Claude Code 通信，提供 6 个工具：

工具名	参数	功能
run_stage	stage: enum	在容器内执行指定 ORFS make 目标
get_metrics	无	读取容器内 metadata.json 全量指标
read_container_file	path: string	读取容器内日志、报告等任意文件
list_files_container	path: string	列出容器内目录内容
read_file	path: string	读取宿主机项目文件
write_file	path, content	写入宿主机文件，用于修改 config.mk

3.3 Skill 编排

项目定义了三个核心 Skill：

backend-flow：主流程编排 Skill，按顺序执行 6 个阶段，每阶段后检查关键指标，根据阈值决策继续/重试/上报：

阶段	成功标准	重试策略
综合	面积 < 50000 μm²，无致命错误	失败则报告 RTL 问题
布图规划	利用率 ≤ 45%	降低 CORE_UTILIZATION
布局	拥塞 < 15%，WNS > -0.5 ns	调整 PLACE_DENSITY
CTS	时钟偏斜 < 0.1 ns	调整缓冲器类型
布线	DRC = 0，WNS ≥ -0.2 ns	启用天线修复
收尾	GDS 文件存在	无需重试

error-diagnosis-retry：错误诊断 Skill，当阶段失败时自动分析日志，按错误类型实施参数级重试，最多重试 2 次。

report_summary.md：报告总结Skill，结合各个步骤生成的log文件，按照指定格式生成关键参数报告，用于直观分析。

3.4 Hooks 自动验证

在 .claude/settings.local.json 中配置 PostToolUse Hooks，每次 Bash 工具调用后自动触发对应验证脚本，用于执行完成后硬代码验证，在检查出错后，返回报错信息，让Agent重新debug运行，直至通过，确保流程正确性：

```
"hooks": {
  "PostToolUse": [{
    "matcher": "Bash",
    "hooks": [
      { "command": ".claude/hooks/validate_synth.sh", "if": "Bash(*1_synth*)" },
      { "command": ".claude/hooks/validate_place.sh", "if": "Bash(*3_place*)" },
      { "command": ".claude/hooks/validate_route.sh", "if": "Bash(*5_route*)" },
      { "command": ".claude/hooks/validate_finish.sh", "if": "Bash(*6_finish*)" }
    ]
  }]
}
```

以 `validate_synth.sh` 为例，验证逻辑包括：

- 1. 检查综合网表文件是否生成
- 2. 扫描综合日志中的 `ERROR` 关键词
- 3. 统计网表中标准单元实例数

若验证失败，Hook 返回 `"continue": false` 并附带诊断信息，阻止 Agent 继续执行后续步骤。

3.5 权限配置

`.claude/settings.local.json` 中明确授权 Agent 可调用的工具，主要为mcp服务和docker操作：

```
"permissions": {
  "allow": [
    "mcp__orfs-agent__run_stage",
    "mcp__orfs-agent__get_metrics",
    "mcp__orfs-agent__read_container_file",
    "mcp__orfs-agent__list_files_container",
    "mcp__orfs-agent__read_file",
    "mcp__orfs-agent__write_file",
    "Bash(docker exec orfs-agent:*)",
    "Bash(docker cp:*)"
  ]
}
```

4. QoR 结果分析

4.1 时序 (Timing)

指标	数值	评价
时钟周期约束	10.0 ns	—
Setup WNS	+0.749 ns	无违规，裕量充足
Setup TNS	0	无违规
Hold WNS	+5.028 ns	无违规

指标	数值	评价
Hold TNS	0	无违规
最大工作频率 (fmax)	108.1 MHz	超过 100 MHz 目标
Setup 违规数	0	—
Hold 违规数	0	—

时序结果优秀。最差路径 WNS = +0.749 ns，说明关键路径延迟约 9.25 ns，在 10 ns 约束下有 7.5% 的裕量。fmax 达到 108.1 MHz，满足设计目标。

4.2 面积 (Area)

指标	数值	评价
Die 面积	90000 μm^2 (300×300)	—
Core 面积	77594.4 μm^2	—
标准单元面积	7778.7 μm^2	利用率仅 10%
逻辑单元数	802	组合逻辑 + 反相器
时序修复 Buffer	117	布线后插入
Fill Cell 数	9128	填充空白区域

利用率 10% 偏低，Die 面积相对于设计规模偏大。对于 ALU 这类小型纯组合逻辑，可将 Die 缩小至 150×150 μm 以提高面积效率，但本次实验以验证流程完整性为主，保守的面积设置有助于避免拥塞问题。

我们又设置了更紧的约束，发现Agent会根据实际返回的结果调整 config.mk 参数，并重新执行流程，最终输出合适结果。

4.3 功耗 (Power)

指标	数值
内部功耗	170.5 μW
开关功耗	256.4 μW
漏电功耗	0.003 μW
总功耗	426.9 μW

开关功耗高于内部功耗，符合纯组合逻辑的特征。总功耗 426.9 μW 在 sky130hd 工艺下属于正常水平。

4.4 布线质量 (Routing)

指标	数值	评价
最终 DRC 违规	0	完全通过
Antenna 违规	0	无天线效应
总 Via 数	6771	—
li1→met1 Via	3286	最多, 符合 sky130 特征
met1→met2 Via	3226	—

DRC 零违规是布线成功的关键指标, 说明 TritonRoute 在当前密度下能够完整完成布线且满足所有设计规则。

4.5 电源完整性 (IR Drop)

网络	平均 IR Drop	最大 IR Drop
VDD	1.14 μ V	8.93 μ V
VSS	1.11 μ V	9.18 μ V

IR Drop 极小 (最大不足 10 μ V), 远低于 sky130hd 的 IR Drop 预算。这得益于较低的利用率和充足的 PDN 网格覆盖。

4.6 PPA 综合评价

维度	结果	评级
时序	fmax = 108.1 MHz, 无违规	优秀
面积	利用率 10%, 面积偏保守	可优化
功耗	426.9 μ W, 正常水平	良好
布线	DRC = 0, Antenna = 0	优秀
IR Drop	最大 9.18 μ V	优秀

附录：关键中间结果

综合统计 (Yosys)

- 综合后标准单元数: 857
- 综合面积: 5589.1 μ m²
- 运行时间: 5.11 s, 峰值内存: 58.9 MB

布图规划

- 行数: 102 rows
- Tap cell 数: 1040
- 核心利用率: 7.2%

布局

- 最终 HPWL: 27693.2 μm
- 镜像优化实例数: 53
- 布局后利用率: 10%

输出文件

文件路径	描述
results/6_final.gds	最终 GDSII 版图
reports/6_report.json	PPA 指标 (JSON)
reports/6_final_netlist.v	后布线门级网表
reports/6_final.spef	寄生参数文件
reports/summary_report.md	流程汇总报告
logs/	各阶段详细日志